

Supple: Flexible and Fast Oblivious Subsampling for Large-Scale Privacy-Preserving Analytics

Xi Luo¹, Zhongming Wang¹, Xiaoguo Li^{1(✉)}, Shengmin Xu², and Tao Xiang¹

¹ Chongqing University, Chongqing, China

luoxi@stu.cqu.edu.cn, {zmwang, csxgli, txiang}@cqu.edu.cn

² Fujian Normal University, Fuzhou, Fujian, China

smxu1989@gmail.com

Abstract. Subsampling is a fundamental primitive in Privacy-Preserving Machine Learning (PPML), especially for mini-batch training. Trusted Execution Environments (TEEs) offer an efficient foundation for PPML, but their software-visible side channels make fully oblivious subsampling necessary. Existing oblivious subsampling algorithms remain impractical at scale: they either incur heavy shuffling overheads or require restrictive relationship among the dataset size, sample size, and number of samples. In this paper, we present **Supple**, a family of fast and flexible oblivious subsampling algorithms. At its core is a compaction-driven binary recursion tree that materializes multiple independent samples in a single execution under flexible parameter settings. To further improve efficiency, **Supple** introduces an offline-precomputed routing tape that decouples routing control from online data movement and avoids propagating large membership metadata during execution. Experimental results show that **Supple** outperforms prior state-of-the-art subsampling algorithms. Specifically, it achieves speedups of $12.78\times$ to $20.35\times$ as the dataset size scales from 2^{20} to 2^{28} .

Keywords: Oblivious Subsampling · Trusted Execution Environment · Privacy-Preserving Machine Learning.

1 Introduction

Privacy-Preserving Machine Learning (PPML) has become a cornerstone of modern data analytics, driven by the escalating need to train and deploy models on sensitive datasets within untrusted cloud environments [2, 19, 25, 30]. Traditional approaches to PPML primarily rely on secure Multi-Party Computation (MPC) or Fully Homomorphic Encryption (FHE) to provide cryptographic security guarantees. However, these techniques often remain impractical for large-scale workloads: FHE is plagued by extreme computational overhead, frequently incurring 10^4 – $10^5\times$ performance penalties compared to plaintext operations [21, 28], while MPC is constrained by prohibitive communication costs and network bandwidth requirements [20, 32]. As a compelling alternative, hardware trust via Trusted Execution Environment (TEE), such as Intel SGX/TDX [3, 11] and

ARM TrustZone [13], has emerged as a practical path forward [4, 14, 18]. By enforcing isolation at the hardware level, TEEs enable sensitive computations to achieve near-native execution efficiency, offering the throughput necessary for PPML on large-scale datasets [7, 10, 18, 31].

Subsampling is a fundamental primitive for scaling modern machine learning, enabling mini-batch Stochastic Gradient Descent (SGD) [8, 15] to train large-scale models. By approximating gradients using random subsets of data, it drastically reduces per-iteration computational and memory overhead, making the training of high-capacity models on massive datasets viable. Beyond efficiency, subsampling acts as a critical “privacy amplifier” within the Differential Privacy (DP) framework [1]. This amplification stems from the stochastic uncertainty of record membership. Since any individual record is included in a batch only with a small probability q , the per-step privacy loss is mathematically tightened from the baseline privacy budget ϵ to $\mathcal{O}(q\epsilon)$ [6]. In iterative ML training, this multiplicative reduction is indispensable; it significantly decelerates the consumption of the cumulative privacy budget, allowing high-utility models to converge over extensive epochs without exhausting formal privacy guarantees.

While TEEs encrypt data in memory, they do not inherently conceal memory access patterns from an untrusted host [27, 29]. Consequently, non-oblivious algorithms fail to maintain execution privacy when running within an enclave [9, 16]. This leakage nullifies the privacy amplification effect of DP by eliminating the stochastic uncertainty inherent in the subsampling process [18, 24]. To restore these theoretical gains, the design of oblivious subsampling is imperative.

Problem Statement. In this work, we study the problem of subsampling with *full obliviousness* [22], meaning that the execution trace reveals nothing about the sample membership of any record. Let $\mathcal{D}$ be a dataset of size n residing within a TEE. Our goal is to design fully oblivious algorithms for the following two settings:

- *Single-batch subsampling:* Given a size m , output a single subset $S \subset \mathcal{D}$ of size m drawn uniformly from all possible subsets.
- *Multi-batch subsampling:* Given size m and count k , output a sequence of k mutually independent samples $\{S_1, \dots, S_k\}$, where each S_i is a single-batch subsample.

1.1 Our Contributions

In this paper, we propose **Supple**, a novel oblivious subsampling framework for TEE-based PPML. **Supple** achieves both high efficiency and broad applicability through a decoupled **offline/online** architecture. At its core, **Supple** precomputes hidden sample memberships in an offline phase and encodes them into a compact *routing tape*, a linearized sequence of routing bits. During the online phase, the enclave consumes this tape to materialize a batch of mutually independent samples via *recursive oblivious compaction* under public size bounds. Our main contributions are summarized as follows:

Flexible multi-batch subsampling. Existing oblivious subsampling algorithm [24] is constrained by a rigid size-matching invariant (e.g., $km = n$), where the data movement logic is strictly hard-coded to the dataset size. This rigidity makes it inefficient or even inapplicable when the sampling parameters (m, k) do not perfectly align with n .

Supple achieves flexibility by decoupling hidden membership from physical output packing. We shift from global data reshuffling to a hierarchical filtering architecture. By separating logical sample requirements from the physical data layout, we introduce public size bounds at each recursion node. This allows the framework to treat subsampling as a hierarchical flow that “retains” only what is potentially needed, enabling a unified execution that is naturally agnostic to the specific ratios of n, m , and k . This flexibility yields strong practical gains: with 16-byte records, $n = 2^{20}$, $m = 2^{14}$, and $k = 2^{12}$ (i.e., $km \neq n$), **Supple** achieves a $34.24\times$ speedup over state-of-the-art multi-batch subsampling [24].

Fast online sample materialization. The primary bottleneck in TEE-based oblivious sampling is the simultaneous management of logic and data movement during the critical online path. Conventional “Tag-and-Shuffle” approaches [24] require the enclave to handle encrypted identifiers for every record and perform multiple global oblivious shuffles to determine their final positions. This “on-the-fly” routing results in heavy computational and memory overhead that hinders scalability for large datasets.

We adopt a principle of computational offloading, transforming online materialization from a complex, decision-heavy reorganization task into a deterministic, instruction-guided flow. By pre-calculating the entire routing path offline and encoding it into a sequential routing tape, we effectively remove the “intelligence” required during the online execution. In this paradigm, the enclave no longer needs to process per-record IDs or perform expensive global shuffles to resolve sample membership. This shift from data-heavy reorganization to a pure data-compaction stream allows **Supple** to achieve excellent performance. With 16-byte records and a sampling rate of $p = 1/64$, **Supple** reduces the online latency to 5.60s per sample for $n = 2^{28}$, outperforming existing state-of-the-art multi-batch subsampling [24] by $12.78\times$.

2 Preliminaries

Notations. We denote the subarray $(D_j, \dots, D_{k-1})$ by $D[j : k]$. For a boolean expression E , let $\mathbf{1}\{E\}$ be its indicator function, which evaluates to 1 if E holds and 0 otherwise.

2.1 Full Obliviousness

Full obliviousness [22] stipulates that an algorithm’s observable execution must not leak information about secret inputs beyond what is revealed by public outputs and input lengths. Unlike weaker notions limited to coarse-grained memory

behavior [18, 26], full obliviousness requires that both the instruction sequence and fine-grained memory access patterns are efficiently simulatable using only public information. This ensures that the execution trace is determined independently of secret values, thereby mitigating side-channel leakage from secret-dependent control flow or micro-architectural patterns.

Follow the definition of full obliviousness given by Sasy et al. [22]. Let $E = \mathcal{E}(A, x_1, \dots, x_k)$ denote the execution of algorithm A on k inputs $(x_1, \dots, x_k)$, which consists of the sequence of executed instructions and the memory contents at every step. We denote its final output as $O(E)$, and the observable execution trace as $\text{Trace}(E) := (\mathcal{M}(E), \mathcal{I}(E))$. Here, $\mathcal{M}(E)$ is the sequence of memory accesses, and $\mathcal{I}(E)$ is the sequence of executed opcodes. Full obliviousness is formally defined as follows, with $[k] = \{1, \dots, k\}$:

Definition 1 (Full Obliviousness). *Let $E = \mathcal{E}(A, x_1, \dots, x_k)$. For any index subset $I \subseteq [k]$, algorithm A is fully oblivious with respect to inputs $\{x_i\}_{i \in I}$ if there exists a PPT simulator S such that for all inputs $(x_1, \dots, x_k)$, the distribution of $\{O(E), \text{Trace}(E)\}$ is indistinguishable to $\{O(E), S((x_i)_{i \notin I}, (|x_i|)_{i \in I})\}$.*

2.2 Oblivious Compaction and Shuffling

Oblivious compaction and shuffling are two fundamental algorithms in oblivious data processing. We recall their formalization as follows:

OCOMPACT. Given a data array $D = (D[0], \dots, D[n-1])$ and a boolean mark array $M \in \{0, 1\}^n$, OCOMPACT partitions D such that all records with mark bit 1 are moved to the front. To ensure full obliviousness, the observable execution trace must be independent of the values in M . The state-of-the-art oblivious compaction algorithm [22] achieves $O(n \log n)$ complexity.

OSHUFFLE. Given an array D of length n , OSHUFFLE applies a uniformly random permutation π to rearrange D in-place, while ensuring the observable trace leaks no information about π beyond the length n . Existing oblivious shuffling algorithms [5, 12, 17, 22, 23] typically follow a tag-then-sort paradigm: assigning random tags to records followed by an oblivious sort.

While state-of-the-art shuffling achieves $O(n \log n)$ complexity, its concrete efficiency is significantly lower than that of OCOMPACT. Recent empirical benchmarks demonstrate that for large-scale datasets ranging from $n = 2^{20}$ to 2^{24} records, oblivious compaction consistently achieves an 8–20× practical speedup over state-of-the-art oblivious shuffling [12, 22, 23]. This advantage stems from a fundamental difference in their computational burden: OCOMPACT effectively performs a 1-bit stable sort, whereas OSHUFFLE must process high-entropy random tags to realize a uniform permutation, incurring substantially larger constant-factor overheads in practice.

3 Supple: Oblivious Subsampling

In this section, we first present the single-batch subsampling algorithm, and then extend it to the multi-batch setting.

Alg. 1: OSBSUBSAMPLE(D, n, m)	Sub-Alg. 1: MARKGEN(n, m)
Input: dataset D of size n . Output: a sample S of size m . ▷ <i>Offline phase</i> 1: $M \leftarrow \text{MARKGEN}(n, m)$ ▷ <i>Online phase</i> 2: $\text{OCOMPACT}(D, M)$ 3: $S \leftarrow D[0 : m]$ 4: $\text{OSHUFFLE}(S)$ 5: return S	Input: dataset size n , sample size m . Output: mark array M of length n , containing exactly m entries equal to 1. 1: Create boolean array M of length n . 2: for $i \leftarrow 0 \dots n - 1$ do 3: Generate $r \in [0, 1)$ uniformly. 4: $M[i] \leftarrow \mathbf{1}\{r < m/(n - i)\}$ 5: $m \leftarrow m - M[i]$ 6: return M

3.1 Single-batch Subsampling

We present OSBSUBSAMPLE (Alg. 1), an efficient single-batch subsampling algorithm that samples a subset of m records from n records. Our design is motivated by the observation that OCOMPACT exhibits significantly superior concrete efficiency compared to OSHUFFLE in practice. Consequently, we prioritize compaction for the bulk of the data movement and restrict the more costly shuffling operation to a small prefix. The algorithm operates in two phases:

- *Offline phase.* MARKGEN (Sub-Alg. 1) takes the total size n and target sample size m to generate a random boolean mark $M \in \{0, 1\}^n$ containing exactly m ones distributed uniformly at random. This mark defines the uniformly chosen subset of records to be sampled.
- *Online phase.* OCOMPACT first obviously moves all marked records to the prefix $D[0 : m]$. Since oblivious compaction fails to fully randomize the selected records, it introduces an inevitable order bias. To achieve complete statistical randomness, we apply OSHUFFLE exclusively to the prefix $D[0 : m]$ to ensure a uniformly random permutation of the selected records.

By focusing the shuffling overhead on a small subset, this design ensures a uniformly random sample of size m while obviating the need to randomize the remaining $n - m$ unselected records.

3.2 Multi-batch Subsampling

Building upon the single-batch subsampling algorithm, we now design RESUBSAMPLE (Alg. 2), an efficient algorithm for multi-batch subsampling that samples k independent subsets of size m from a dataset of size n .

A naive approach to multi-batch subsampling involves repeatedly executing the single-batch algorithm k times. However, this iterative method introduces substantial redundancy, as each execution obviously scans the entire dataset even though only a small fraction of records is ultimately retained. For large n , the total overhead scales linearly with k , making it prohibitive for high-throughput applications. We thus require a “one-shot” approach that can sample all k subsets while minimizing redundant data movement.

Alg. 2: RESUBSAMPLE(D, M, m, k)

Input: dataset D of size n , number of samples k , membership array M generated in the *offline phase*.

Output: a tuple $\mathcal{S} = (S_0, \dots, S_{k-1})$ consisting of k samples, each of size m .

```

1:  $n \leftarrow |D|$ 
2: if  $k = 1$  then
3:    $S \leftarrow D[0 : m]$ 
4:   OSHUFFLE( $S$ )
5:   return ( $S$ )
6:  $k_l \leftarrow \lfloor k/2 \rfloor, k_r \leftarrow k - k_l$ 
7: Create  $n$ -bit arrays  $B_l, B_r$ .
8: for  $i \leftarrow 0 \dots n - 1$  do
9:    $B_l[i] \leftarrow \mathbf{1}\{M[i] \& ((1 \ll k_l) - 1) \neq 0\}$ 
10:   $B_r[i] \leftarrow \mathbf{1}\{(M[i] \gg k_l) \& ((1 \ll k_r) - 1) \neq 0\}$ 
11:  $t_l \leftarrow \min(n, m \cdot k_l), t_r \leftarrow \min(n, m \cdot k_r)$ 
     $\triangleright$  compact inputs for the subproblems
12:  $D_l \leftarrow \text{COMPACTD}(D, B_l, t_l)$ 
13:  $D_r \leftarrow \text{COMPACTD}(D, B_r, t_r)$ 
14:  $M_l \leftarrow \text{COMPACTM}(M, B_l, t_l, 0, k_l)$ 
15:  $M_r \leftarrow \text{COMPACTM}(M, B_r, t_r, k_l, k_r)$ 
     $\triangleright$  recursively solve the two subproblems
16:  $\mathcal{S}_l \leftarrow \text{RESUBSAMPLE}(D_l, M_l, m, k_l)$ 
17:  $\mathcal{S}_r \leftarrow \text{RESUBSAMPLE}(D_r, M_r, m, k_r)$ 
18: return ( $\mathcal{S}_l \parallel \mathcal{S}_r$ )

```

Sub-Alg. 2: VECMARKGEN(n, m, k)

Input: dataset size n , sample size m , number of samples k .

Output: membership array M .

```

1:  $M \leftarrow$  length- $n$  array filled with 0
2:  $C \leftarrow$  length- $k$  array filled with  $m$ 
3: for  $i \leftarrow 0 \dots n - 1$  do
4:   for  $j \leftarrow 0 \dots k - 1$  do
5:     Generate  $r \in [0, 1)$  uniformly.
6:      $b \leftarrow \mathbf{1}\{r < C[j]/(n - i)\}$ 
7:      $M[i] \leftarrow M[i] \mid (b \ll j)$ 
8:      $C[j] \leftarrow C[j] - b$ 
9: return  $M$ 

```

Sub-Alg. 3: COMPACTM(M, B, t, s, ℓ)

```

1: Copy  $M$  and store it in  $M'$ .
2: OCOMPACT( $M', B$ )
3: for  $i \leftarrow 0 \dots t - 1$  do
4:    $temp_1 \leftarrow M'[i] \gg s$ 
5:    $temp_2 \leftarrow (1 \ll \ell) - 1$ 
6:    $M'[i] \leftarrow temp_1 \& temp_2$ 
7: return  $M'[0 : t)$ 

```

Sub-Alg. 4: COMPACTD(D, B, t)

```

1: Copy  $D$  and store it in  $D'$ .
2: OCOMPACT( $D', B$ )
3: return  $D'[0 : t)$ 

```

Intuition. We structure the subsampling process as a *binary recursion tree* (see Fig. 1). The root node begins with the full dataset, while each leaf corresponds to a final sample. At each internal node, records are filtered and propagated only to the relevant child subtrees. By discarding irrelevant records at the earliest possible stage, each node operates on a progressively smaller candidate set.

The recursive algorithm. Our recursive framework follows the aforementioned offline/online paradigm:

- *Offline phase:* VECMARKGEN (Sub-Alg. 2) generalizes the boolean mark array of the single-batch case into a k -bit membership array M . For each record $D[i]$, the entry $M[i]$ is a k -bit word where the j -th bit is set if and only if $D[i]$ belongs to sample S_j . Note that a record may simultaneously belong to multiple target samples.

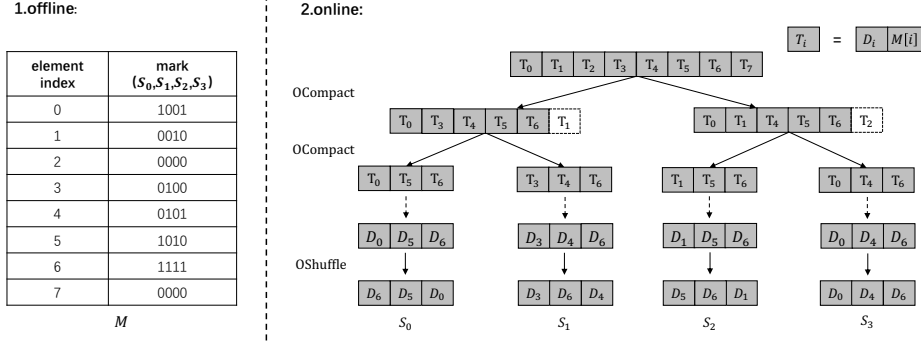


Fig. 1: Illustration of oblivious multi-batch subsampling for $n = 8, k = 4$, and $m = 3$. Each record D_i is initially bundled with a k -bit membership vector $M[i]$. At the root, OCOMPACT routes tuples to child subtrees based on their target sample membership. To maintain obliviousness, each internal node is allocated a fixed-size buffer (here, $\min(8, 2 \cdot 3) = 6$), with unused slots padded by dummy tuples (dashed boxes). This recursive filtering continues until the leaf nodes, where the selected $m = 3$ records are compacted to the prefix. Finally, OSHUFFLE is applied to these prefixes to yield the final randomized samples.

- *Online phase:* RESUBSAMPLE (Alg. 2) processes the dataset D guided by M . At each internal node, the algorithm generates two boolean routing arrays, B_l and B_r , which identify records required by the left and right child subtrees, respectively. These records are obviously packed to the front using OCOMPACT, while irrelevant records are truncated before propagation. Finally, since OCOMPACT cannot guarantee a completely random permutation of the selected records (introducing an order bias), OSHUFFLE is applied exclusively at the leaf nodes to ensure the uniform randomness of each final m -sized sample.

This hierarchical pruning ensures that child nodes receive only the records necessary for their assigned subproblems, decoupling the execution cost from any fixed global relation between n and km . This design provides remarkable flexibility, seamlessly accommodating various parameter regimes (e.g., $n \gg k \cdot m$ or $n < k \cdot m$) without the rigid data replication required by the prior work [24].

Obliviousness. To guarantee full obliviousness, the size of each child subproblem must remain strictly independent of the actual number of records routed to it. For any node at recursion depth i (where the root is $i = 0$ and $n_0 = n$), we allocate a fixed-size buffer of length $n_i = \min(n_{i-1}, m \cdot \lceil k/2^i \rceil)$. Here, the assigned sample count $\lceil k/2^i \rceil$ is deterministically derived from the public total k and the current depth i . Routed records are obviously compacted to the front of this buffer, with any remaining capacity padded by dummy entries. While this padding introduces localized overhead, it is vastly outweighed by the volume of irrelevant records pruned during recursion. Consequently, this fixed-buffer strat-

Alg. 3: OMBSUBSAMPLE(D, n, m, k) Input: dataset D of size n, number of samples k. Output: a tuple $\mathcal{S} = (S_0, \dots, S_{k-1})$ consisting of k samples, each of size m. ▷ <i>Offline phase</i> 1: $M \leftarrow \text{VECMARKGEN}(n, m, k)$ 2: $n_B \leftarrow \text{ROUTEBITNUM}(n, m, k)$ 3: Create an array B of length n_B. 4: $p_1 \leftarrow \text{REPRECOM}(M, B, m, k, 0)$ 5: assert $p_1 = n_B$ ▷ <i>Online phase</i> 6: $(\mathcal{S}, p_2) \leftarrow \text{RESAMPLE}(D, B, m, k, 0)$ 7: assert $p_2 = n_B$ 8: return $\mathcal{S}$	Sub-Alg. 5: ROUTEBITNUM(n, m, k) Input: dataset size n, sample size m, number of samples k. Output: the number of routing bits n_B. 1: if $k = 1$ then 2: return 0 3: $k_l \leftarrow \lfloor k/2 \rfloor$ 4: $k_r \leftarrow k - k_l$ 5: $t_l \leftarrow \min(n, m \cdot k_l)$ 6: $t_r \leftarrow \min(n, m \cdot k_r)$ 7: $n_{B,l} \leftarrow \text{ROUTEBITNUM}(t_l, m, k_l)$ 8: $n_{B,r} \leftarrow \text{ROUTEBITNUM}(t_r, m, k_r)$ 9: $n_B \leftarrow 2n + n_{B,l} + n_{B,r}$ 10: return n_B
Sub-Alg. 6: REPRECOM(M, B, m, k, p) Input: membership array M, routing tape B, sample size m, number of samples k, starting position p. Output: next free position p. 1: $n \leftarrow M$ 2: if $k = 1$ return p 3: $k_l \leftarrow \lfloor k/2 \rfloor$, $k_r \leftarrow k - k_l$ 4: Create n-bit arrays B_l, B_r. 5: for $i \leftarrow 0 \dots n - 1$ do 6: $B_l[i] \leftarrow \mathbf{1}\{M[i] \& ((1 \ll k_l) - 1) \neq 0\}$ 7: $B_r[i] \leftarrow \mathbf{1}\{(M[i] \gg k_l) \& ((1 \ll k_r) - 1) \neq 0\}$ 8: $B[p : p + n] \leftarrow B_l$, $p \leftarrow p + n$ 9: $B[p : p + n] \leftarrow B_r$, $p \leftarrow p + n$ 10: $t_l \leftarrow \min(n, m \cdot k_l)$, $t_r \leftarrow \min(n, m \cdot k_r)$ 11: $M_l \leftarrow \text{COMPACTM}(M, B_l, t_l, 0, k_l)$ 12: $M_r \leftarrow \text{COMPACTM}(M, B_r, t_r, k_l, k_r)$ 13: $p \leftarrow \text{REPRECOM}(M_l, B, m, k_l, p)$ 14: $p \leftarrow \text{REPRECOM}(M_r, B, m, k_r, p)$ 15: return p	Sub-Alg. 7: RESAMPLE(D, B, m, k, p) Input: dataset D, routing tape B, sample size m, number of samples k, starting position p. Output: a tuple $\mathcal{S} = (S_0, \dots, S_{k-1})$ consisting of k samples, each of size m, next unread position p. 1: $n \leftarrow D$ 2: if $k = 1$ then 3: $S \leftarrow D[0 : m]$ 4: $\text{OSHUFFLE}(S)$ 5: return (S, p) 6: $k_l \leftarrow \lfloor k/2 \rfloor$, $k_r \leftarrow k - k_l$ 7: $B_l \leftarrow B[p : p + n]$, $p \leftarrow p + n$ 8: $B_r \leftarrow B[p : p + n]$, $p \leftarrow p + n$ 9: $t_l \leftarrow \min(n, m \cdot k_l)$, $t_r \leftarrow \min(n, m \cdot k_r)$ 10: $D_l \leftarrow \text{COMPACTD}(D, B_l, t_l)$ 11: $D_r \leftarrow \text{COMPACTD}(D, B_r, t_r)$ 12: $(S_l, p) \leftarrow \text{RESAMPLE}(D_l, B, m, k_l, p)$ 13: $(S_r, p) \leftarrow \text{RESAMPLE}(D_r, B, m, k_r, p)$ 14: return $(S_l \parallel S_r, p)$

egy ensures that all memory access patterns and execution paths depend solely on public parameters (e.g., n, m, k), rigorously satisfying the formal requirements of full obliviousness without compromising the algorithm's overall efficiency.

3.3 Memory-Friendly Multi-batch Subsampling

While the baseline recursive design avoids the redundancy of repeated single-batch executions, it introduces a new inefficiency: the tight coupling of the

membership metadata M with the online recursive execution. In practice, this coupling creates two primary bottlenecks that hinder performance at scale:

- *Substantial space overhead.* Each record must carry a k -bit membership word throughout the recursion. The algorithm must repeatedly propagate, compact, and reshape this metadata alongside the data itself. For a large k , this metadata dwarfs the actual payload. For instance, with a record size $\beta = 16$ bytes and $k = 1024$, the 128-byte membership word accounts for nearly 89% of the total per-record payload, drastically increasing the effective volume of every OCOMPACT operation.
- *Redundant online control-processing.* At every internal node, the algorithm must rescan the membership array to derive child routing arrays (B_l and B_r) and compact the membership entries for child subproblems. This layer-by-layer reconstruction of control information, rather than direct consumption, incurs significant computational friction and degrades memory locality.

Intuition. To resolve these bottlenecks, we observe that the recursive routing schedule is entirely independent of the actual data records; it depends exclusively on the initial membership array M . This insight allows us to fully decouple the control plane (routing decisions) from the data plane (record movement).

Based on this decoupling, we replace the heavy online propagation of M with a compact global *routing tape* B . Conceptually similar to an audio cassette tape, B is an append-only structure that is “recorded” (written) exactly once and later “played back” (read sequentially). We replace the heavy online propagation of M with a compact, global *routing tape* B , which is an append-only structure that is “recorded” (written) once and “played back” (read sequentially) later. This design is inherently memory-friendly: by stripping the massive k -bit membership metadata from the online data path, the effective payload size reverts to the original record size β . Consequently, the oblivious compaction primitives operate on “bare” records, significantly reducing the memory footprint.

The tape-based algorithm. Under this design, the execution flow seamlessly maintains our established offline/online paradigm, as formalized in OMBSUBSAMPLE (Alg. 3):

- *Offline phase:* The primary objective of this stage is to construct the global routing tape B . First, ROUTEBITNUM (Sub-Alg. 5) calculates the required total length of the tape to allocate memory. Then, REPRECOM (Sub-Alg. 6) traverses the recursion tree using only the initial membership array M . At each internal node, it derives the B_l and B_r routing arrays, appends them directly to B , and recursively compacts the surviving membership entries. As a result, this phase successfully compiles all complex, dynamic control-flow decisions into the final routing tape B —a single, contiguous bit-array ready for sequential playback.
- *Online phase:* Implemented by RESAMPLE (Sub-Alg. 7), this stage processes the actual dataset D without ever seeing M . At each internal node, the algorithm simply reads the next precomputed segment from the tape to serve as the routing signal. It then applies COMPACTD to materialize child inputs of

public sizes $t_l = \min(n_v, m \cdot k_l)$ and $t_r = \min(n_v, m \cdot k_r)$. As the tape pointer advances deterministically alongside the fixed recursion schedule, the need for on-the-fly control extraction is entirely eliminated. Upon reaching a leaf node, a final local OSHUFFLE removes any residual order bias. Ultimately, this tape-based design transforms complex, metadata-heavy recursive processing into a high-performance sequence of “read-and-move” operations.

Efficiency. The above design is memory-friendly. By stripping the massive k -bit membership metadata away from the online data path, the effective payload size drops dramatically back to the original record size β . Consequently, the oblivious compaction primitives operate on bare records rather than inflated composite tuples, which drastically reduces the memory footprint and bandwidth pressure.

4 Analysis

In this section, we analyze the correctness, obliviousness, and theoretical performance of **Supple**.

4.1 Correctness

We begin by establishing the correctness of the single-batch algorithm.

Theorem 1 (Correctness of OSBSample). *OSBSAMPLE(D, n, m) (Alg. 1) outputs a sample of size m drawn uniformly without replacement from the input dataset D .*

For the multi-batch architecture, correctness hinges on the alignment between the offline and online phases. Since OMBSAMPLE utilizes a global routing tape to synchronize these phases, we first establish *tape consistency*, ensuring that the sequence of routing bits consumed during online data materialization precisely matches the sequence generated during offline precomputation. Lemma 1 guarantees this consistency by demonstrating that both phases deterministically traverse an identical, data-independent recursion tree.

Lemma 1 (Tape Consistency). *Let $n_B = \text{ROUTEBITNUM}(n, m, k)$ (Sub-Alg. 5) and B be a global boolean array of length n_B . Let $\mathcal{W} = (w_0, \dots, w_{n_B-1})$ denote the sequence of routing bits written by **REPRECOM**($M, B, m, k, 0$) (Sub-Alg. 6), $\mathcal{R} = (r_0, \dots, r_{n_B-1})$ be the sequence consumed by **RESAMPLE**($D, B, m, k, 0$) (Sub-Alg. 7). For any valid input D and membership array M , both recursive executions terminate such that their tape pointers satisfy $p_1 = p_2 = n_B$ and the bit sequences are identical, i.e., $\mathcal{W} = \mathcal{R}$.*

Given the tape consistency established in Lemma 1, the online phase accurately enforces the sample memberships predetermined offline. Since **VECMARKGEN** assigns these memberships independently and uniformly at random for each of the k batches, the final materialized output rigorously preserves these statistical properties. We formalize the overall correctness of the multi-batch algorithm in Theorem 2.

Theorem 2 (Correctness of OMBSubsample). *OMBSUBSAMPLE(D, n, m, k) (Alg. 3) outputs a tuple $\mathcal{S} = (S_0, \dots, S_{k-1})$ consisting of k mutually independent samples, each of size m and each drawn uniformly without replacement from the input dataset D .*

4.2 Obliviousness

We now demonstrate that our proposed algorithms satisfy full obliviousness, ensuring that their observable execution traces are independent of secret data.

For single-batch subsampling, full obliviousness is a direct consequence of the composition of its underlying algorithms. The offline phase, MARKGEN, operates exclusively on the public parameters (n, m) and internal randomness. Similarly, the online phase invokes OCOMPACT and OSHUFFLE using fixed, public bounds. Since all control-flow decisions and memory-access sequences are derived solely from public parameters and are completely decoupled from the actual dataset contents, the execution trace remains invariant regardless of the secret input.

Theorem 3. *OSBSUBSAMPLE(D, n, m) (Alg. 1) is fully oblivious with respect to D .*

Extending this guarantee to the multi-batch architecture requires verifying that the dynamic recursive routing does not introduce side-channel leakage. In our design, the entire recursion tree topology and all intermediate subproblem sizes are statically determined by the public parameters (n, m, k) . Furthermore, as established by Lemma 1, the offline and online phases interact solely via the fixed-size routing tape B . Because the online phase consumes this tape in a data-oblivious manner—without dynamically reconstructing control flow—and all data movements are handled by the oblivious OCOMPACT primitive, the observable trace remains perfectly deterministic and data-independent.

Theorem 4. *OMBSUBSAMPLE(D, n, m, k) (Alg. 3) is fully oblivious with respect to D .*

4.3 Performance

We finally analyze the asymptotic complexity of the proposed algorithms. We assume that the underlying algorithms, OCOMPACT and OSHUFFLE, are implemented using state-of-the-art designs [12, 17, 22] which achieve $O(s \log s)$ time and $O(1)$ auxiliary workspace for an input of size s . For detailed analysis of these theorems, please refer to Appendix A.

For OSBSUBSAMPLE, the offline phase traverses the dataset of size n exactly once to generate random bits and deduct quotas, resulting in an optimal $O(n)$ time and $O(n)$ -bit memory trace. Subsequently, the online phase applies exactly one full-sized OCOMPACT and one local OSHUFFLE of size m , requiring strictly $O(n \log n)$ and $O(m \log m)$ time. Since the transient space involves only the dataset itself, it amounts to exactly $O(n)$ records. Based on this analysis, we establish Theorem 5.

Theorem 5 (Performance of OSBSubsample). *Let n be the dataset size and m be the sample size ($m \leq n$). The runtime and storage cost of OSBSUBSAMPLE (D, n, m) (Alg. 1) are as follows:*

1. *The offline phase runs in $O(n)$ time and requires $O(n)$ bits.*
2. *The online phase runs in $O(n \log n + m \log m)$ time, with $O(n)$ peak transient record storage.*

In practice scenarios where $m \ll n$ (e.g., mini-batch SGD optimization), the online phase is dominated by the $O(n \log n)$ full-dataset compaction. Consequently, a naive iterative approach to generate k independent batches would incur a total online complexity of $O(k \cdot n \log n)$.

Theorem 6 (Runtime of OMBSubsample). *Let n be the dataset size, m be the sample size ($m \leq n$), and k be the target number of samples. The runtime of OMBSUBSAMPLE (D, n, m, k) (Alg. 3) is as follows:*

1. *The offline phase runs in*

$$O\left(\begin{cases} nk + n \log n + mk \log k \log n, & mk \leq n, \\ nk + mk(1 + \log(n/m)) \log n, & mk > n \end{cases}\right) \text{ time.}$$

2. *The online phase runs in*

$$O\left(\begin{cases} n \log n + mk \log k \log n + km \log m, & mk \leq n, \\ mk(1 + \log(n/m)) \log n + km \log m, & mk > n \end{cases}\right) \text{ time.}$$

Theorem 6 explicitly demonstrates the flexibility of our architecture: the execution cost dynamically adapts to the arbitrary relationship between the dataset size n and the total sample demand mk . Furthermore, in the typical regime where $m \ll n$, our approach yields massive theoretical speedups over the naive $O(k \cdot n \log n)$ baseline. Specifically, when $m \cdot k \leq n$, the online complexity is effectively bounded by $O(n \log n \log k)$, achieving a speedup factor of $\mathcal{O}(k / \log k)$. Conversely, in the heavy-sampling regime ($m \cdot k > n$), the online cost becomes $O(mk \log(n/m) \log n)$, achieving a speedup factor of $\mathcal{O}\left(\frac{n/m}{\log(n/m)}\right)$.

Theorem 7 (Storage of OMBSubsample). *Let n be the dataset size, m be the sample size ($m \leq n$), and k be the target number of samples. The storage cost of OMBSUBSAMPLE (D, n, m, k) (Alg. 3) is as follows:*

1. *The offline routing tape B requires*

$$O\left(\begin{cases} n + mk \log k, & mk \leq n, \\ mk(1 + \log(n/m)), & mk > n \end{cases}\right) \text{ bits.}$$

2. *The online phase incurs a peak transient storage of*

$$O\left(\begin{cases} n, & mk \leq n, \\ n(1 + \log(mk/n)), & mk > n \end{cases}\right) \text{ records.}$$

As shown in Theorem 7, our multi-batch architecture introduces a slight logarithmic increase in peak online storage when $m \cdot k > n$ compared to the $O(n)$ memory of the iterative single-batch approach. However, this minor trade-off is outweighed by its superior metadata efficiency.

Theorem 8 (Routing Tape vs. Membership Array). *Let $|M| = nk$ denote the bit-length of the explicit membership array in the baseline recursive algorithm, and $|B| = n_B$ denote the number of routing bits generated by ROUTEBITNUM (n, m, k) (Sub-Alg. 5). Assume that k and n/m are powers of two. Then:*

1. *If $mk \leq n$, then*

$$n_B = 2n + 2mk(\log_2 k - 1).$$

Consequently, $n_B \leq nk$ for all $k \geq 4$, with equality only in the boundary cases $k = 2$, or $k = 4$ together with $n = mk$.

2. *If $mk > n$, then*

$$n_B = 2mk(1 + \log_2(n/m)) - 2n.$$

Consequently, $n_B < nk$ whenever $n/m \geq 8$, whereas $n_B > nk$ when $n/m \in \{1, 2, 4\}$ (with the sole exception of $n = m$ and $k = 2$, where $n_B = nk$).

By decoupling the control flow from the data path, the routing tape avoids the prohibitive overhead of attaching k -bit membership vectors to each record. Theorem 8 formalizes this advantage, demonstrating that our routing-tape optimization is more memory-efficient than the baseline across most practical parameter regimes, thereby validating our design’s efficacy.

5 Experiments

All our experiments are conducted on an Alibaba Cloud server equipped with an 8-core 2.7 GHz Intel Xeon Scalable (Ice Lake) processor. The system supports Intel SGXv2 with 32GB of protected memory. All reported results are obtained from a single-threaded implementation. Our implementation is publicly available at <https://github.com/luoxi1228/Supple.git>.

5.1 Experimental Setup

Baselines. We evaluate three variants of Supple introduced in Section 3: OSBSUBSAMPLE, RESUBSAMPLE, and OMBSUBSAMPLE. These are benchmarked against two state-of-the-art baselines: the single-subsampling *Shuffle-and-Truncate* (S&T) method and the multi-batch subsampling scheme (PSQF) by Sasy et al. [24]. To ensure a fair comparison, we implement the underlying OCOMPACT and OSHUFFLE algorithms using [22] across all evaluated algorithms.

Parameters. The performance of Supple is affected by four parameters: dataset size n , block size β , sampling rate $p = m/n$, and the number of samples k . All timings are reported on a per-sample basis, except for experiments explicitly varying k . For multi-batch algorithms, we follow the PSQF convention by setting $k = n/m = 1/p$, reporting the average per-sample time (total time divided by k). In all comparative experiments against the baselines, we report the online execution time for all Supple variants.

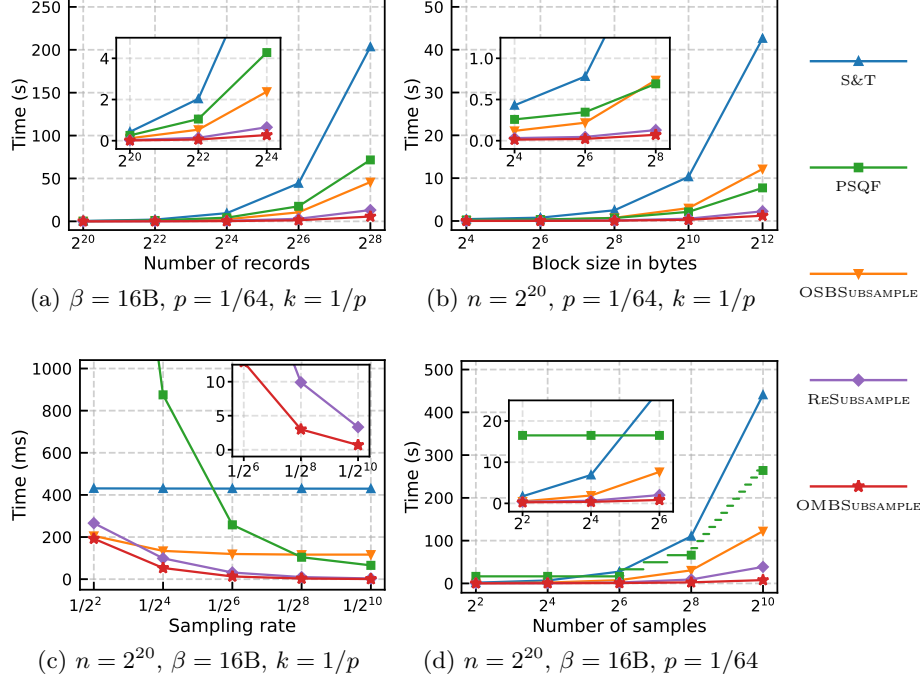


Fig. 2: Performance of different subsampling algorithms under different parameter settings. (a), (b), and (c) report the average per-sample sampling time, while (d) reports the total time to generate all k samples. Specifically, (a) varies the dataset size n , (b) varies the block size β , (c) varies the sampling rate p , and (d) varies the number of samples k .

5.2 Experimental Results

Below, we show that **Supple** outperforms the corresponding baselines across all evaluated settings. Its advantage is most pronounced in batch-intensive workloads, where shifting more work offline substantially shortens the online phase.

Impact of Dataset Size and Block Size. Fig. 2a and 2b illustrate the per-sample execution time as dataset size n and block size β increase. While runtimes naturally grow across all schemes, our **RESUBSAMPLE** and **OMBSUBSAMPLE** consistently attain the lowest runtimes and the most gradual growth. Notably, **PSQF** is initially slower than our **OSBSUBSAMPLE**, indicating the massive constant-factor penalty of oblivious shuffling compared to lightweight compaction. Moreover, a crossover occurs in Fig. 2b around $\beta = 2^8$ bytes, where **PSQF** eventually overtakes **OSBSUBSAMPLE**. This intersection reflects a key trade-off: for small blocks, compaction’s lower overhead outweighs the cost of

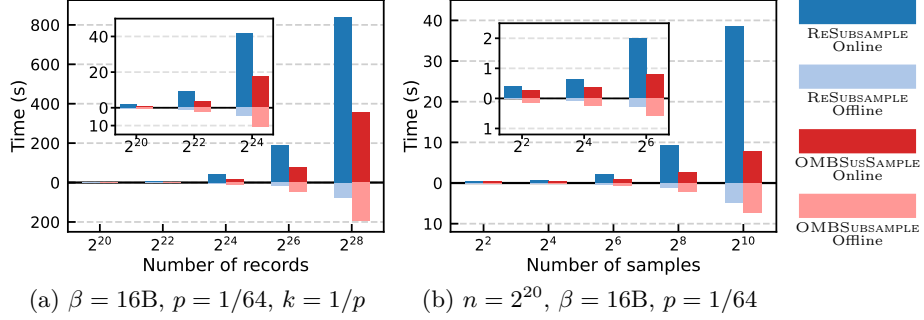


Fig. 3: Offline and online runtime breakdown of RESUBSAMPLE and OMBSUBSAMPLE under varying dataset size n and number of samples k .

OSBSUBSAMPLE’s k repeated passes; for large blocks, the resulting data movement eventually dominates and makes PSQF more favorable.

Bypassing this trade-off entirely, our tape-based OMBSUBSAMPLE utilizes fast compaction while completely eliminating redundant data passes, yielding remarkable scalability across all regimes. Specifically, it achieves a $12.78\text{--}20.35\times$ speedup over PSQF and an $8.17\text{--}9.24\times$ speedup over OSBSUBSAMPLE when n scales from 2^{20} to 2^{28} . Similar commanding leads ($6.27\text{--}20.11\times$ over PSQF and $9.30\text{--}10.68\times$ over OSBSUBSAMPLE) are observed when β scales from 16B to 4KB. Compared to our intermediate recursive baseline RESUBSAMPLE, OMBSUBSAMPLE yields a smaller yet relatively stable improvement ($2.35\text{--}2.42\times$ across varying n , and $1.80\text{--}2.45\times$ across varying β), as both schemes share the identical recursive backbone and scale similarly with data volume.

Impact of Sampling Rate. Fig. 2c evaluates the per-sample latency across varying sampling rates p (with $k = 1/p$). As p decreases, the batched schemes become substantially faster. In contrast, S&T and OSBSUBSAMPLE remain static, as full-dataset shuffles and compactions dominate their execution regardless of the sampling rate. OMBSUBSAMPLE consistently performs the best, with its latency plummeting from 192.17 ms at $p = 1/4$ to just 0.68 ms at $p = 1/1024$. A lower sampling rate entails a smaller sample size m , which reduces the required capacity per branch, enabling upper levels to aggressively prune irrelevant data and rapidly shrink the downward processing volume. Consequently, the speedup of OMBSUBSAMPLE peaks in the small- p regime, reaching up to $171.39\times$ over OSBSUBSAMPLE, $4.88\times$ over RESUBSAMPLE, and $96.39\times$ over PSQF.

Impact of the Number of Samples. Fig. 2d reports the total runtime required to generate k samples. While the total time naturally increases with k for all methods, OMBSUBSAMPLE remains the most efficient from $k = 4$ to $k = 1024$. It achieves up to a $15.85\times$ speedup over OSBSUBSAMPLE, $5.01\times$ over RESUBSAMPLE, and $63.46\times$ over PSQF. Notably, as shown in the figure, PSQF exhibits a distinct step-wise growth curve. This behavior stems from its architectural rigidity: PSQF is fundamentally constrained to partition the dataset into a

fixed number of samples per execution ($k = 1/p$). In stark contrast, OMBSUBSAMPLE scales smoothly and continuously across all three parameter regimes ($km < n$, $km = n$, and $km > n$). The widening performance gap confirms that our unified single-pass execution adapts seamlessly to diverse batch volumes, entirely bypassing the rigid structural bottlenecks that penalize prior works [24] and demonstrating superior flexibility.

Impact of Tape-Based Optimization. Fig. 3 visually dissects the execution time into offline and online components to demonstrate the effectiveness of OMBSUBSAMPLE over RESUBSAMPLE. Although OMBSUBSAMPLE intentionally incurs a heavier offline penalty to precompute the routing tape, this strategic shift pays massive dividends in the online phase. By decoupling the control logic and stripping the bulky k -bit membership metadata from the online data path, OMBSUBSAMPLE accelerates the online phase by a stable $2.35\text{--}2.42\times$ across all dataset sizes n (Fig. 3a). Because the online savings vastly outweigh the offline penalty, the total execution time is consistently reduced by over $1.6\times$.

The advantage of OMBSUBSAMPLE becomes remarkably pronounced as the number of samples k increases (Fig. 3b). As k scales up, the online execution time of RESUBSAMPLE increases drastically, peaking at 38.60s for $k = 1024$. In contrast, OMBSUBSAMPLE effectively restricts this online runtime to a mere 7.71s, thereby achieving a $5.01\times$ online speedup and a $2.89\times$ overall speedup compared to RESUBSAMPLE. This confirms that our routing tape successfully resolves the bottleneck of multi-batch scaling.

6 Conclusion

In this paper, we presented **Supple**, a family of fully oblivious subsampling algorithms for efficient single/multi-batch sample generation. The design is built on two key ideas: a compaction-driven binary recursion tree for batched sample materialization, and an offline-precomputed routing tape that separates routing control from online data movement. Together, they support arbitrary relationships among the dataset size, sample size, and number of samples, while avoiding the shuffle-heavy cost and rigid parameter constraints of prior work. By decoupling control metadata from records, **Supple** further reduces online overhead without sacrificing full obliviousness. Both our analysis and experiments show that **Supple** improves over prior approaches, achieving excellent flexibility together with strong practical performance.

References

1. Abadi, M., Chu, A., Goodfellow, I., et al.: Deep learning with differential privacy. In: ACM Conference on Computer and Communications Security (CCS) (2016)
2. Al-Rubaie, M., Chang, J.M.: Privacy-preserving machine learning: Threats and solutions. In: IEEE Symposium on Security and Privacy (S&P) (2019)
3. Anati, I., Gueron, S., Johnson, S., et al.: Innovative technology for cpu based attestation and sealing. In: International Workshop on Hardware and Architectural Support for Security and Privacy (HASP) (2013)

4. Apple: Private cloud compute: A new frontier for ai privacy in the cloud (2024)
5. Asharov, G., Chan, T.H.H., Nayak, K., et al.: Bucket oblivious sort: An extremely simple oblivious sort. In: SIAM Symposium on Simplicity in Algorithms (SOSA) (2020)
6. Balle, B., Barthe, G., Gaboardi, M.: Privacy amplification by subsampling: Tight analyses via couplings and divergences. In: Conference on Neural Information Processing Systems (NeurIPS) (2018)
7. Bittau, A., Erlingsson, Ú., Maniatis, P., et al.: Prochlo: Strong privacy for analytics in the crowd. In: ACM Symposium on Operating Systems Principles (SOSP) (2017)
8. Bottou, L.: Large-scale machine learning with stochastic gradient descent. In: Proceedings of COMPSTAT'2010 (2010)
9. Brasser, F., Müller, U., Dmitrienko, A., et al.: Software grand exposure: Sgx cache attacks are practical. In: USENIX Workshop on Offensive Technologies (WOOT) (2017)
10. Chen, H.H., Chen, H., Sun, M., et al.: Picachv: Formally verified data use policy enforcement for secure data analytics. In: USENIX Security Symposium (USENIX Security) (2025)
11. Cheng, P.C., Ozga, W., Valdez, E., et al.: Intel tdx demystified: A top-down approach. *ACM Computing Surveys* (2024)
12. Gu, T., Wang, Y., Tinoco, A., et al.: Flexway o-sort: Enclave-friendly and optimal oblivious sorting. In: USENIX Security Symposium (USENIX Security) (2025)
13. Hua, Z., Gu, J., Xia, Y., et al.: vtz: virtualizing {ARM}{TrustZone}. In: USENIX Security Symposium (USENIX Security) (2017)
14. Huang, P., Hoang, T., Li, Y., et al.: Efficient privacy-preserving machine learning with lightweight trusted hardware. *Proceedings on Privacy Enhancing Technologies (PoPETs)* (2024)
15. LeCun, Y., Bengio, Y., Hinton, G.: Deep learning. *Nature* (2015)
16. Lee, S., Shih, M.W., Gera, P., et al.: Inferring fine-grained control flow inside sgx enclaves with branch shadowing. In: USENIX Security Symposium (USENIX Security) (2017)
17. Ngai, N., Demertzis, I., Chamani, J.G., et al.: Distributed & scalable oblivious sorting and shuffling. In: IEEE Symposium on Security and Privacy (S&P) (2024)
18. Ohrimenko, O., Schuster, F., Fournet, C., et al.: Oblivious multi-party machine learning on trusted processors. In: USENIX Security Symposium (USENIX Security) (2016)
19. Pang, Q., Zhu, J., Möllering, H., et al.: Bolt: Privacy-preserving, accurate and efficient inference for transformers. In: IEEE Symposium on Security and Privacy (S&P) (2024)
20. Ruan, W., Lin, X., Zhou, R., et al.: Hawkeye: Statically and accurately profiling the communication cost of models in multi-party learning. In: USENIX Security Symposium (USENIX Security) (2025)
21. Samardzic, N., Feldmann, A., Krastev, A., et al.: F1: A fast and programmable accelerator for fully homomorphic encryption. In: IEEE/ACM International Symposium on Microarchitecture (MICRO) (2021)
22. Sasy, S., Johnson, A., Goldberg, I.: Fast fully oblivious compaction and shuffling. In: ACM Conference on Computer and Communications Security (CCS) (2022)
23. Sasy, S., Johnson, A., Goldberg, I.: Waks-on/waks-off: Fast oblivious offline/online shuffling and sorting with waksman networks. In: ACM Conference on Computer and Communications Security (CCS) (2023)
24. Sasy, S., Ohrimenko, O.: Oblivious sampling algorithms for private data analysis. In: Conference on Neural Information Processing Systems (NeurIPS) (2019)

25. Song, L., Zhang, Q., Lin, Y., et al.: Suda: An efficient and secure unbalanced data alignment framework for vertical {Privacy-Preserving} machine learning. In: USENIX Security Symposium (USENIX Security) (2025)
26. Tinoco, A., Gao, S., Shi, E.: Enigmap: External-memory oblivious map for secure enclaves. In: USENIX Security Symposium (USENIX Security) (2023)
27. Van Bulck, J., Weichbrodt, N., Kapitza, R., et al.: Telling your secrets without page faults: Stealthy page table-based attacks on enclaved execution. In: USENIX Security Symposium (USENIX Security) (2017)
28. Viand, A., Jattke, P., Haller, M., Hithnawi, A.: Heco: Fully homomorphic encryption compiler. In: USENIX Security Symposium (USENIX Security) (2023)
29. Xu, Y., Cui, W., Peinado, M.: Controlled-channel attacks: Deterministic side channels for untrusted operating systems. In: IEEE Symposium on Security and Privacy (S&P). pp. 640–656 (2015)
30. Yuan, B., Yang, S., Zhang, Y., et al.: Md-ml: Super fast privacy-preserving machine learning for malicious security with a dishonest majority. In: USENIX Security Symposium (USENIX Security) (2024)
31. Zheng, W., Dave, A., Beekman, J.G., et al.: Opaque: An oblivious and encrypted distributed analytics platform. In: USENIX Symposium on Networked Systems Design and Implementation (NSDI) (2017)
32. Zheng, W., Deng, R., Chen, W., et al.: Cerebro: A platform for multi-party cryptographic collaborative learning. In: USENIX Security Symposium (USENIX Security) (2021)

A Appendix

Let n denote the dataset size, m the sample size, and k the total number of target samples. Let level $i \in \{0, \dots, \log_2 k\}$ denote the current depth of the recursion tree, where the root is at $i = 0$. At level i , there are 2^i nodes, and each node is responsible for generating $k/2^i$ samples. According to the public size bounding rule, the buffer size for any node at level i is $t_i = \min(n, \frac{m \cdot k}{2^i})$.

Proof of Theorem 6 (Runtime of OMBSubsample)

Proof. Because the recursion topology and buffer sizes $t_i = \min(n, \frac{m \cdot k}{2^i})$ depend strictly on public parameters, both the offline REPRECOM and the online RESAMPLE phases execute identically structured recursions. We first define the aggregated compaction volume V as the total number of records processed by OCOMPACT across all internal nodes (from level $i = 0$ to level $\log_2 k - 1$, noting that leaf nodes do not perform compaction).

Case 1 ($m \cdot k \leq n$): The root node (level 0) processes n items. For all subsequent internal levels $i \in \{1, \dots, \log_2 k - 1\}$, the node capacity adheres to the demand-driven bound $t_i = \frac{m \cdot k}{2^i}$. Since there are 2^i nodes at level i , each level collectively processes $2^i \times (\frac{m \cdot k}{2^i}) = m \cdot k$ items. Summing the root and the $\log_2 k - 1$ subsequent levels yields the total volume $V = n + m \cdot k(\log_2 k - 1) < n + m \cdot k \log_2 k$.

Case 2 ($m \cdot k > n$): The capacity t_i transitions from the parent-bound n to the demand-driven bound $\frac{m \cdot k}{2^i}$ at the crossover level $i^* = \log_2(m \cdot k/n)$. For the upper levels $i \in \{0, \dots, i^* - 1\}$, $t_i = n$. The total number of nodes in

these upper levels follows a geometric series: $\sum_{i=0}^{i^*-1} 2^i = 2^{i^*} - 1$. Substituting $2^{i^*} = m \cdot k / n$, the aggregated volume for the upper tree is exactly $n(m \cdot k / n - 1) = m \cdot k - n$. For the lower internal levels $i \in \{i^*, \dots, \log_2 k - 1\}$, $t_i = \frac{m \cdot k}{2^i}$. Each level uniformly processes a volume of $m \cdot k$. The total number of such levels is $(\log_2 k - 1) - i^* + 1 = \log_2(n/m)$. Thus, the aggregated volume for the lower tree is exactly $m \cdot k \log_2(n/m)$. Combining both parts, the total volume is $V = (m \cdot k - n) + m \cdot k \log_2(n/m)$, which is upper-bounded by $V < m \cdot k (1 + \log_2(n/m))$.

Final Complexity Derivation: Let s_j denote the input size of the j -th internal node across the recursion tree, such that the aggregated volume $V = \sum s_j$. The oblivious compaction primitive requires $O(s_j \log s_j)$ time per node. Since the maximum node size is bounded by the original dataset size ($s_j \leq n$), we can establish a strict upper bound for the total compaction cost:

$$\sum O(s_j \log s_j) \leq \sum O(s_j \log n) = O\left(\sum s_j \log n\right) = O(V \log n).$$

This mathematically justifies that processing an aggregated volume V in fragmented pieces of size $s_j \leq n$ is bounded by $\frac{V}{n} O(n \log n) = O(V \log n)$. Substituting our derived V into $O(V \log n)$ provides the foundational cost: If $m \cdot k \leq n$, $O(V \log n) = O(n \log n + m \cdot k \log_2 k \log n)$; If $m \cdot k > n$, $O(V \log n) = O(m \cdot k (1 + \log_2(n/m)) \log n)$; Finally, we differentiate the two phases by adding their exclusive operations: (1) Offline Phase: Requires an initial $O(n \cdot k)$ time to construct the explicit membership array via VECMARKGEN. Adding $O(n \cdot k)$ to $O(V \log n)$ precisely yields the offline expressions stated in the theorem. (2) Online Phase: Omits the initialization but strictly requires $O(k \cdot m \log m)$ time to apply OSHUFFLE locally at the k leaf nodes. Adding $O(k \cdot m \log m)$ to $O(V \log n)$ exactly yields the online expressions stated in the theorem.

Proof of Theorem 7 (Storage of OMBSsample)

Proof. Assuming a standard depth-first traversal (DFS) of the recursion tree, the peak transient storage at any given moment equals the sum of array sizes along the longest active path from the root down to a leaf. The length of this path is exactly $\log_2 k + 1$ nodes (from level $i = 0$ to level $\log_2 k$).

Case 1 ($m \cdot k \leq n$): The root node allocates n records. Every subsequent node on the path at level $i \geq 1$ allocates exactly $t_i = \frac{m \cdot k}{2^i}$ records. Using the geometric series sum formula, the total memory along the path is:

$$H = n + \sum_{i=1}^{\log_2 k} \frac{m \cdot k}{2^i} = n + m \cdot k \left(1 - \left(\frac{1}{2}\right)^{\log_2 k}\right) = n + m \cdot k \left(1 - \frac{1}{k}\right) = n + m \cdot k - m.$$

Since $m \cdot k \leq n$, the maximum path sum is strictly $H < 2n$. Consequently, the peak transient storage is bounded by exactly $O(n)$.

Case 2 ($m \cdot k > n$): The path traverses through the dense upper tree until the crossover level $i^* = \log_2(\frac{m \cdot k}{n})$. For the first $i^* + 1$ levels ($i \in \{0, \dots, i^*\}$), the buffer size remains n , yielding a sub-sum of exactly $n(i^* + 1) = n(1 +$

$\log_2(\frac{m \cdot k}{n})$). For the remaining lower levels $i \in \{i^*+1, \dots, \log_2 k\}$, the size geometrically halves starting from $\frac{m \cdot k}{2^{i^*+1}} = n/2$. Summing these geometrically decreasing buffers yields $\frac{n}{2} + \frac{n}{4} + \dots + m$, which strictly sums to $n - m < n$. Combining both segments, the peak memory is strictly bounded by $H < n(1 + \log_2(\frac{m \cdot k}{n})) + n = n(2 + \log_2(\frac{m \cdot k}{n}))$. Dropping the constants yields $O(n(1 + \log_2(\frac{m \cdot k}{n})))$.

Offline Tape Memory: Every internal node writes exactly 2 routing bits per processed record. Thus, the total tape length n_B is strictly twice the aggregated internal volume V computed in Theorem 6. The asymptotic bounds logically mirror those of $O(V)$.

Proof of Theorem 8 (Routing Tape vs. Membership Array)

Proof. The total number of generated routing bits n_B corresponds to assigning 2 bits to each record processed by all internal nodes. Mathematically, this is evaluated as $n_B = 2 \times \sum_{i=0}^{\log_2 k-1} (2^i \times t_i)$.

Case 1 ($m \cdot k \leq n$): The root level requires $2n$ bits. For the remaining internal levels, $t_i = \frac{m \cdot k}{2^i}$, resulting in:

$$n_B = 2n + 2 \sum_{i=1}^{\log_2 k-1} 2^i \left(\frac{m \cdot k}{2^i} \right) = 2n + 2 \sum_{i=1}^{\log_2 k-1} m \cdot k = 2n + 2m \cdot k(\log_2 k - 1).$$

To prove $n_B \leq n \cdot k$, we use the condition $m \cdot k \leq n$. Substituting n with $m \cdot k$ in the first term provides an upper bound: $n_B \leq 2m \cdot k + 2m \cdot k(\log_2 k - 1) = 2m \cdot k \log_2 k$. We then solve for $2m \cdot k \log_2 k \leq n \cdot k$, which simplifies to $2 \log_2 k \leq n/m$. Since $m \cdot k \leq n$ intrinsically implies $k \leq n/m$, the inequality holds as long as $2 \log_2 k \leq k$. This algebraic property holds true for all $k \geq 4$. In the boundary case $k = 2$, we have $n_B = 2n$, whereas $n \cdot k = 2n$, resulting in equality.

Case 2 ($m \cdot k > n$): We split the summation at the crossover level $i^* = \log_2(m \cdot k/n)$:

$$n_B = 2 \left(\sum_{i=0}^{i^*-1} 2^i \cdot n + \sum_{i=i^*}^{\log_2 k-1} 2^i \left(\frac{m \cdot k}{2^i} \right) \right)$$

Evaluating the first geometric sum yields $2n(2^{i^*} - 1)$. Substituting $2^{i^*} = \frac{m \cdot k}{n}$, this equates to $2m \cdot k - 2n$. The second sum aggregates a constant $m \cdot k$ across $\log_2 k - i^*$ levels, yielding exactly $m \cdot k \log_2(n/m)$. Multiplying by 2 and adding both parts results in: $n_B = 2m \cdot k(1 + \log_2(n/m)) - 2n$. To mathematically determine when $n_B < n \cdot k$, let $x = n/m$. We evaluate the condition $2m \cdot k(1 + \log_2 x) \leq m \cdot k \cdot x$, which simplifies to solving $2(1 + \log_2 x) \leq x$. Testing the powers of two: for $x = 8$, the equation becomes $2(1+3) = 8 \leq 8$. At this point, $2m \cdot k(1 + \log_2(n/m)) = n \cdot k$, and since n_B subtracts $2n$, we strictly have $n_B = n \cdot k - 2n < n \cdot k$. Because logarithmic growth is slower than linear growth, $n_B < n \cdot k$ fundamentally holds for all $n/m \geq 8$. Conversely, testing $x \in \{1, 2, 4\}$ yields $2(1 + \log_2 x) > x$, forcing the leading term of n_B to heavily outweigh $n \cdot k$. Algebraic expansion uniquely identifies $x = 1$ ($n = m$) and $k = 2$ as the singular edge case where $n_B = 2m \times 2 \times 1 - 2m = 2m = n \cdot k$.